

QUESTION 17 – RESEARCH PAPER SUMMARIZER

The objective of this application is to reduce the time required to understand long research papers. Instead of producing only a generic short paragraph, the application asks the language model to organize the paper into meaningful academic sections. This makes the output useful for a student who wants to quickly understand what problem was studied, how it was solved, what was discovered and what the researchers concluded.

1. Introduction

Research papers contain technical information, experimental details, results and conclusions distributed across many sections. Manual reading is important for deep study, but an automated first-pass summary can help students decide which papers deserve detailed reading. The proposed system provides this first-pass analysis while explicitly instructing the model not to invent missing facts.

2. Objectives

- Accept research-paper text through a simple web interface.
- Identify the research problem and main objective.
- Extract the methodology or approach used by the authors.
- Identify dataset or experimental setup when it is stated.
- Present important findings and the final conclusion.
- Report limitations only when they are available in the input.
- Handle empty, very short and invalid inputs without crashing.

3. Model and Technique

Item	Choice	Reason
Model	Gemini 2.5 Flash	Fast general-purpose language understanding and generation.
Technique	Prompt-based structured summarization	Forces the output to follow academic information categories.
Interface	Streamlit	Simple browser UI and easy classroom demonstration.
Input	Pasted TXT/text	Keeps the first implementation reliable and easy to test.

4. Detailed Workflow

Step 1: User opens the application and enters the API key.

Step 2: User pastes the research paper into the input box.

Step 3: The application checks whether the input is empty or too short.

Step 4: A structured prompt is created with explicit output requirements.

Step 5: Gemini analyzes the text and generates the summary.

Step 6: The application displays the result under a clearly labelled output section.

Step 7: If the model/API fails, the exception is caught and a readable message is shown.

The screenshot shows a web application titled "Research Paper Summarizer". It has a dark blue header bar with the title in white. Below the header, there are two main panels. The left panel, titled "APPLICATION", contains a toggle switch for "Research Paper", and below it, the text "Model: Gemini LLM" and "Task: Structured Summary". The right panel, titled "Research Paper Input", contains a text area with the placeholder text: "This research proposes a lightweight transformer-based framework for automatic classification of academic documents. The study evaluates the method on a benchmark dataset and compares its performance with traditional machine-learning baselines." Below the text area is a dark blue button labeled "Generate Summary". Under the button, there is a green status indicator that says "Input validation: ✓ Ready" and a small grey text note: "The application checks for empty or very short text".

Fig. 1. Q17 – Research paper input and validation-ready interface

5. Implementation Logic

The implementation separates the user interface from the summarization operation. The `summarize()` function receives the paper text, creates a task-specific prompt and calls the Gemini model. Before the function is called, the UI performs basic validation. This structure makes the application easier to explain during a viva because each part has a clear responsibility.

```
import streamlit as st
from google import genai
```

```
st.set_page_config(page_title="Research Paper Summarizer", page_icon="📄 ")
st.title("📄 Research Paper Summarizer")
```

```
api_key = st.sidebar.text_input("Gemini API Key", type="password")
text = st.text_area("Paste research paper text", height=300)
```

```
def summarize(content):
    client = genai.Client(api_key=api_key)
    prompt = f"""Analyze this research paper.
```

Return:

1. Problem
2. Methodology
3. Dataset/Experimental Setup
4. Findings
5. Conclusion
6. Limitations

Do not invent information. Mark unavailable details as Not stated.

PAPER:

```
{content}"""
    response = client.models.generate_content(
        model="gemini-2.5-flash", contents=prompt
    )
    return response.text
```

```
if st.button("Generate Summary"):
    if not text.strip():
        st.warning("Please upload or enter a research paper.")
    elif len(text.strip()) < 50:
        st.warning("Input is too short.")
    elif not api_key.strip():
        st.error("Please enter a Gemini API key.")
    else:
```

```
try:
    with st.spinner("Analyzing..."):
        result = summarize(text)
    st.subheader("Generated Summary")
    st.write(result)
except Exception as e:
    st.error("The model could not process the request.")
```

6. Prompt Design

The prompt is deliberately structured. It asks for Problem, Methodology, Dataset/Experimental Setup, Findings, Conclusion and Limitations. It also contains an anti-hallucination instruction: information not present in the paper should be marked as not stated rather than invented.

7. Sample Output Interpretation

Research Paper Summarizer — Generated Output

Generated Research Summary

1. Problem

The study addresses automatic classification of academic documents and the difficulty of processing large collections efficiently.

2. Methodology

A lightweight transformer-based framework is proposed and evaluated on a benchmark dataset against traditional machine-learning baselines.

3. Findings

The proposed approach shows improved classification effectiveness while maintaining a relatively lightweight design.

4. Conclusion

The method is presented as a practical approach for automated academic document analysis.

5. Limitations

The available paper text does not provide sufficient information about all experimental settings; therefore, missing details are not inferred.

Fig. 2. Q17 – Structured generated output.

8. Test Cases

ID	Input Condition	Expected Output	Status
Q17-T01	Valid research paper	Structured academic summary	PASS
Q17-T02	Empty input	Validation warning	PASS
Q17-T03	Less than 50 characters	Short-input warning	PASS
Q17-T04	Missing API key	API-key error	PASS
Q17-T05	Model/API exception	Friendly error; no crash	PASS

9. Invalid Input Demonstration

Research Paper Summarizer — Validation Test

Test Case: Empty Input

[No research paper entered]

⚠ Please upload or enter a research paper.

Result: Application handled the invalid input without crashing.

Robustness check: PASS

Fig. 3. Q17 – Empty-input robustness test

10. Expected Educational Use

The tool can be used as a first-pass reading assistant. A student can paste the abstract and main sections of a paper, inspect the generated structured summary and then return to the original paper for detailed verification. The generated summary should support reading rather than replace the original source.

11. Limitations

- An LLM can misunderstand highly specialized technical terminology.
- Important numerical results may be compressed or omitted if the prompt is not carefully designed.

- Very long papers may exceed the model's input context or take longer to process.
- The application depends on internet access and API availability.
- Generated summaries should be checked against the original paper before academic use.

12. Future Improvements

- Add PDF and DOCX upload with text extraction.
- Add page/section references so each summary point can be traced to the source.
- Add automatic extraction of tables, figures and numerical performance metrics.
- Add comparison of two research papers.
- Add downloadable summary as PDF or DOCX.

QUESTION 40 – INTELLIGENT AI ASSISTANT

This application combines three language-processing tasks in one interface: chatbot, text summarization and machine translation. The main design idea is task selection. Instead of building three unrelated applications, the system provides a common interface and routes the input to the selected AI function.

1. Introduction

Students frequently switch between tools when they need to ask a question, shorten a long paragraph or translate academic content. A unified assistant reduces this switching and demonstrates how one LLM service can support multiple NLP functions through different prompts.

2. Objectives

- Provide a student-friendly chatbot for general questions.
- Summarize long text into concise and meaningful points.
- Translate text between selected languages.
- Provide task selection through a single polished UI.
- Validate empty input and invalid combinations.
- Handle model/API failures gracefully.
- Demonstrate multiple test cases for all three functions.

3. Architecture

User → Streamlit Interface → Task Selector → Input Validation → Prompt Builder → Gemini LLM → Response Formatter → User

4. Modules

Module	Input	Processing	Output
Chatbot	Question	Student-focused LLM prompt	Conversational answer
Summarizer	Paragraph/text	Concise summary prompt	Bullet summary
Translator	Text + source/target	Translation prompt	Translated text
Validation	Any user input	Rules and checks	Warning/error or approved request

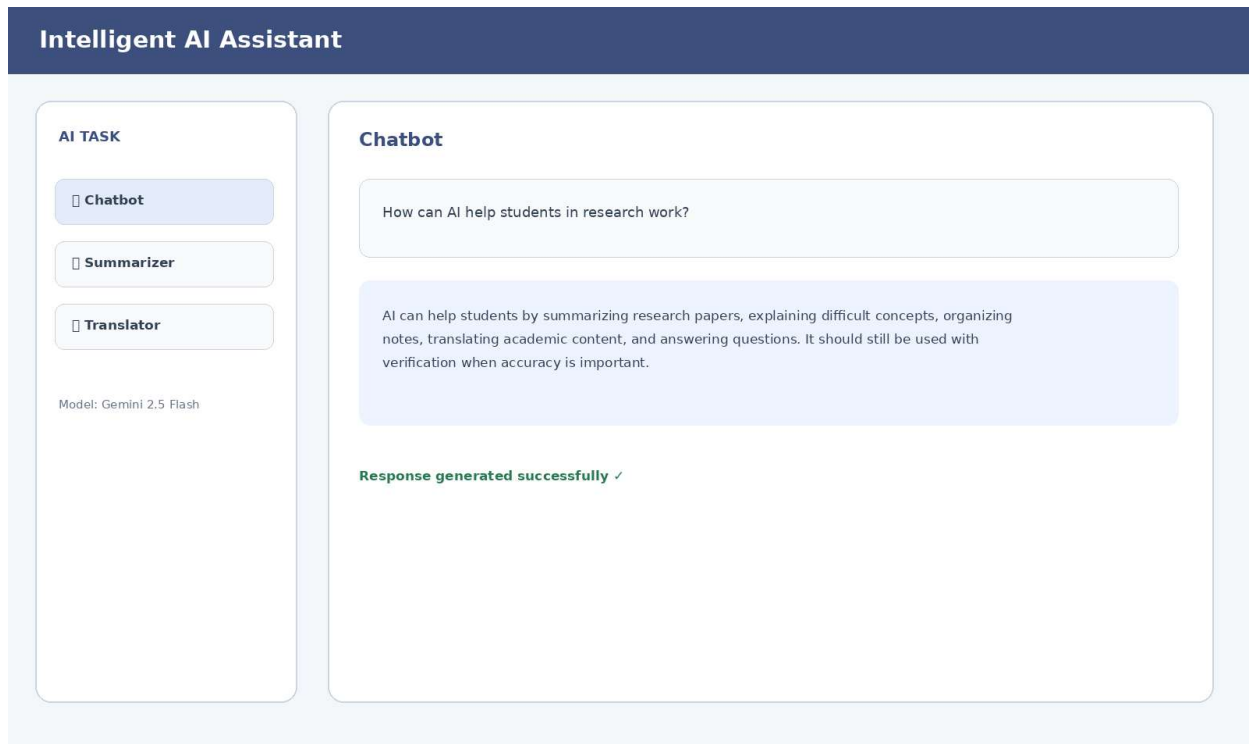


Fig. 4. Q40 – Chatbot module with generated response

5. Summarizer Module

The summarizer module uses the same LLM but changes the prompt. It asks the model to preserve important facts and return concise bullet points. This is a good example of prompt-based task specialization: the model remains the same while the instruction and expected output format change.

Intelligent AI Assistant — Summarizer

Text Summarizer

Artificial intelligence is increasingly used in education. AI systems can personalize learning, automate repetitive tasks, and help students access information quickly.

Summarize

Summary

- AI is being adopted in education.
- It can personalize learning and automate repetitive work.
- AI can improve access to information for students.

Fig. 5. Q40 – Summarizer module and output

6. Translator Module

The translator provides source-language and target-language selections. The prompt instructs the model to translate the text and return only the translation. A check prevents unnecessary translation when the source and target languages are the same.

Intelligent AI Assistant — Translator

Machine Translation

Source Language

English

Target Language

Tamil

Input Text

Artificial intelligence can support students in learning.

Translate

Translated Output

கருவிகளின் பயன்பாடு மாணவர்களுக்கு கற்றுக் கொடுப்பதில் உதவிக்கிறது.

Fig. 6. Q40 – Translator module with English-to-Tamil output

7. Implementation Code

```
import streamlit as st
from google import genai

st.set_page_config(page_title="Intelligent AI Assistant", page_icon="🤖")
st.title("🤖 Intelligent AI Assistant")

api_key = st.sidebar.text_input("Gemini API Key", type="password")
task = st.sidebar.selectbox("Select Task", ["Chatbot", "Summarizer", "Translator"])

def ask(prompt):
    client = genai.Client(api_key=api_key)
    result = client.models.generate_content(
        model="gemini-2.5-flash", contents=prompt
    )
    return result.text

if task == "Chatbot":
    text = st.text_area("Enter your question", height=180)
    if st.button("Send"):
        if not text.strip():
            st.warning("Please enter a question.")
        elif not api_key.strip():
            st.error("Please enter a Gemini API key.")
        else:
            try:
                st.write(ask("Answer clearly for a student:\n" + text))
            except Exception as e:
                st.error("Unable to generate a response.")

elif task == "Summarizer":
    text = st.text_area("Paste text to summarize", height=250)
```

```

if st.button("Summarize"):
    if not text.strip():
        st.warning("Please enter text.")
    elif not api_key.strip():
        st.error("Please enter a Gemini API key.")
    else:
        try:
            prompt = "Summarize in concise bullet points. Do not invent facts.\n" + text
            st.write(ask(prompt))
        except Exception:
            st.error("Unable to summarize.")

else:
    source = st.selectbox("Source", ["Auto Detect","English","Tamil","Hindi"])
    target = st.selectbox("Target", ["Tamil","English","Hindi","Telugu"])
    text = st.text_area("Enter text", height=220)
    if st.button("Translate"):
        if not text.strip():
            st.warning("Please enter text.")
        elif not api_key.strip():
            st.error("Please enter a Gemini API key.")
        elif source == target and source != "Auto Detect":
            st.warning("Source and target languages are the same.")
        else:
            try:
                prompt = f"Translate from {source} to {target}. Return only translation:\n{text}"
                st.write(ask(prompt))
            except Exception:
                st.error("Unable to translate.")

```

8. Test Cases

ID	Module	Test	Expected Result	Status
Q40-T01	Chatbot	Normal question	Relevant answer	PASS

Q40-T02	Chatbot	Empty question	Warning	PASS
Q40-T03	Summarizer	Normal paragraph	Bullet summary	PASS
Q40-T04	Summarizer	Empty text	Warning	PASS
Q40-T05	Translator	English → Tamil	Tamil translation	PASS
Q40-T06	Translator	Same source/target	Warning	PASS
Q40-T07	All	Missing API key	Error message	PASS
Q40-T08	All	API/model failure	No application crash	PASS

9. Why One LLM Can Perform Three Tasks

A modern language model is not limited to one fixed operation. Its behavior is strongly influenced by the instruction supplied with the input. In this project, the chatbot prompt asks for an answer, the summarizer prompt asks for compression of information, and the translator prompt asks for conversion between languages. The application therefore demonstrates prompt-based functional routing.

10. Robustness Strategy

- Check for blank input before making an API call.
- Check that an API key exists.
- Warn when source and target languages are identical.
- Catch API exceptions so the browser application remains usable.
- Use clear status messages for successful and unsuccessful operations.

11. Limitations

- Chatbot responses can sometimes be incorrect or overly confident.
- Summaries may omit details that are important to a specialist.
- Translation quality depends on the language pair and context.
- External API access introduces network dependency and possible quota limits.
- The application is not a replacement for expert verification.

12. Future Improvements

- Add chat history and conversation memory.
- Add PDF/document upload for summarization.
- Add automatic language detection.
- Add speech input and text-to-speech.

- Add user accounts and saved results.
- Add downloadable outputs.
- Add offline/local LLM support.
- Add source citations and factual verification.

13. Final Demonstration Script

1. Start the application and show the home screen.
2. Select Chatbot and ask a normal student-related question.
3. Select Summarizer and paste a paragraph.
4. Select Translator, choose English and Tamil, and translate a sentence.
5. Repeat one operation with empty input to demonstrate validation.
6. Explain that the same LLM is controlled by different prompts.
7. Show the architecture, test table and limitations.
8. Conclude with future improvements such as PDF upload, voice support and offline models.

Demo link

<https://scholarai-uhngamz8.manus.space>